DG INTERNS HUB

Cybersecurity Internship Program

WEEK 6

WEB SECURITY & VULNERABILITY ASSESSMENT AND PENETRATION TESTING

Prepared By: Prakhar Gupta

Role: Cybersecurity Intern

Submission Date: 14/Oct/2026

1. Introduction to VAPT

Vulnerability Assessment and Penetration Testing (VAPT) is a structured security-testing practice used to identify, evaluate, and address weaknesses in an application, network, or system before they can be exploited by attackers. It combines two complementary approaches:

- Vulnerability Assessment (VA): a systematic review of a system to identify, classify, and prioritize known vulnerabilities, typically using automated scanning tools.
- Penetration Testing (PT): a simulated, hands-on attack against the system to actively exploit vulnerabilities and determine the real-world impact of a successful breach.

For web applications specifically, VAPT focuses on the application layer — how the app handles user input, sessions, authentication, and server configuration. The Open Web Application Security Project (OWASP) maintains the widely-referenced "OWASP Top 10", a list of the most critical and commonly exploited web application security risks, which forms the theoretical basis for this project.

This report documents a week-long, lab-based exercise in which known-vulnerable practice applications (DVWA and a public test site) were used to study, reproduce, and document common vulnerability classes in a safe, legal, and controlled environment, along with the tools and methodology used to detect them.

Why VAPT Matters

- Identifies security gaps before malicious actors do.
- Helps organizations meet compliance requirements (ISO 27001, PCI-DSS, GDPR, etc.).
- Reduces the risk of data breaches, financial loss, and reputational damage.
- Provides actionable, prioritized remediation guidance to development teams.

Scope of This Project

Testing in this project was performed exclusively against Damn Vulnerable Web Application (DVWA), an application intentionally built with security flaws for training purposes, running in a local/isolated lab environment, with an optional exercise against the public legal test target testphp.vulnweb.com. No production system or third-party application without authorization was tested.

2. OWASP Top 10 — Vulnerabilities Studied

The OWASP Top 10 is refreshed periodically based on real-world breach data and industry surveys. Four categories were studied in depth for this project, summarized below.

2.1 SQL Injection (SQLi)

SQL Injection occurs when untrusted user input is inserted into a SQL query without proper validation or parameterization, allowing an attacker to alter the query's logic. It is categorized under OWASP's broader "Injection" risk class.

How it works

A vulnerable application concatenates user input directly into a SQL statement. If the input is not sanitized, an attacker can inject additional SQL syntax that changes the query's meaning — for example, bypassing a login check or extracting data from other tables.

Example (illustrative — for a login form)

```
Normal query:  SELECT * FROM users WHERE username = '$user' AND password = '$pass';If
'admin' --  is entered as the username, the resulting query becomes:  SELECT * FROM
users WHERE username = 'admin' -- ' AND password = '';(everything after -- is treated
as a SQL comment, so the password check is skipped)
```

Impact

- Unauthorized access to sensitive data (credentials, PII, financial records).
- Authentication bypass.
- Data modification or deletion (in severe cases).
- In some configurations, remote command execution on the database server.

Common variants

- In-band SQLi (Union-based, Error-based).
- Blind SQLi (Boolean-based, Time-based).
- Out-of-band SQLi.

2.2 Cross-Site Scripting (XSS)

XSS occurs when an application includes untrusted data in a web page without proper validation or escaping, allowing an attacker to run arbitrary JavaScript in another user's browser session.

Types of XSS

- Reflected XSS — the malicious script is part of the request (e.g. a URL parameter) and is immediately reflected back in the response; it typically requires tricking a victim into clicking a crafted link.
- Stored XSS — the malicious script is saved on the server (e.g. in a comment or profile field) and served to every user who views that content, making it more dangerous since no victim interaction with a link is needed.
- DOM-based XSS — the vulnerability exists in client-side JavaScript that unsafely handles data from the DOM, without the payload necessarily touching the server.

Example payload (illustrative, harmless alert)

```
<script>alert('XSS')</script>
```

Impact

- Session hijacking via cookie theft.
- Defacement of web pages.
- Redirection to phishing or malware sites.
- Keystroke logging and credential theft within the browser context.

2.3 Broken Authentication

This category covers flaws in how an application manages user identity, login, session tokens, and credential handling, allowing attackers to compromise passwords, keys, or session tokens, or exploit implementation flaws to assume other users' identities.

Common weaknesses

- Weak or default credentials permitted (e.g. admin/admin).
- No protection against automated credential stuffing or brute-force attacks (no rate limiting / account lockout).
- Session IDs exposed in the URL, or not invalidated after logout.
- Missing or weak multi-factor authentication.
- Passwords stored in plaintext or with weak hashing (e.g. unsalted MD5).

Impact

- Full account takeover.
- Privilege escalation if an administrative account is compromised.
- Identity spoofing and impersonation.

2.4 Security Misconfiguration

Security misconfiguration is one of the most commonly observed issues and arises when security settings are not defined, implemented, or maintained correctly, or when defaults are left unchanged.

Common examples

- Default accounts and passwords left enabled.
- Directory listing enabled, exposing file structure.
- Verbose error messages that leak stack traces or database details.
- Unnecessary services, ports, or features left running.
- Missing security headers (e.g. Content-Security-Policy, X-Frame-Options, Strict-Transport-Security).
- Outdated software or unpatched components.

Impact

- Information disclosure that aids further attacks.
- Unauthorized administrative access.
- Increased attack surface across the whole application.

3. Tools Used

3.1 Burp Suite (Community Edition)

Burp Suite is an integrated platform for web application security testing. In this project it was used primarily as an intercepting proxy to capture, inspect, and modify HTTP requests and responses exchanged between the browser and DVWA.

- Proxy: intercepts browser traffic for manual inspection and payload tampering.
- Repeater: resend and modify a captured request repeatedly to test different payloads.
- Target / Site map: visualize the application's structure as it is browsed.
- Decoder: encode/decode data (URL, Base64, etc.) found in requests.

3.2 OWASP ZAP (Zed Attack Proxy)

OWASP ZAP is a free, open-source dynamic application security testing (DAST) tool. It was used to run automated crawling and active-scanning against DVWA to identify vulnerabilities without manual payload crafting.

- Automated Scan: spiders the target and runs a library of active attack rules against discovered endpoints.
- Alerts panel: lists identified issues, grouped by risk (High/Medium/Low/Informational).
- HTML/PDF report export: produces a shareable scan report.

3.3 DVWA (Damn Vulnerable Web Application)

DVWA is a PHP/MySQL web application deliberately built with a range of well-known vulnerabilities, intended for legal, hands-on practice of common attack techniques in a controlled environment. It offers adjustable security levels (Low / Medium / High / Impossible) so the same vulnerability can be tested against increasingly robust defenses.

3.4 Environment Setup Summary

Component	Setup Method	Purpose
XAMPP	Apache + MySQL + PHP stack installed locally	Hosts DVWA on localhost

DVWA	Cloned from GitHub into XAMPP htdocs, database created via setup.php	Target vulnerable application
Component	**Setup Method**	**Purpose**
Burp Suite CE	Downloaded from PortSwigger, browser proxy set to 127.0.0.1:8080	Intercept & analyze traffic
OWASP ZAP	Downloaded from zaproxy.org	Automated vulnerability scanning
Browser (Firefox)	FoxyProxy extension used to switch proxy on/off	Traffic routed through Burp/ZAP



Figure 1. DVWA Security Level Configuration and Setup Verification

4. Testing Process (DVWA)

4.1 SQL Injection Testing

Module used: DVWA → SQL Injection. Security level: Low.

Step 1 — Baseline request

A normal User ID (e.g. 1) was submitted to observe expected behavior and the resulting query response.

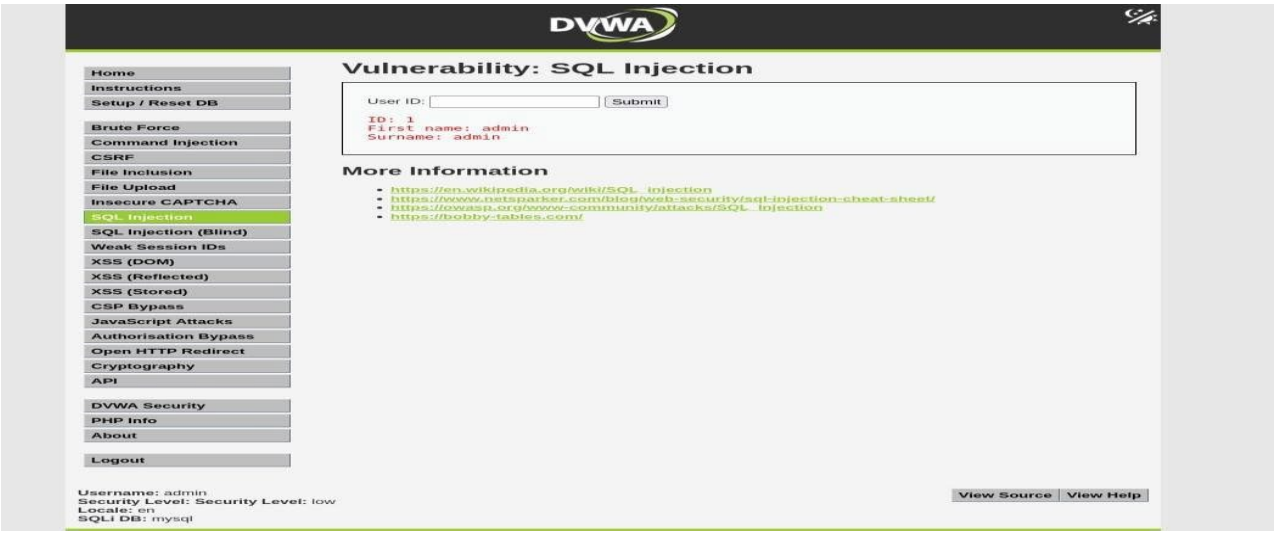


Figure 2. DVWA SQL Injection module showing the baseline response for User ID 1.

Step 2 — Payload injection

The input field was tested with classic authentication-bypass / data-extraction payloads:

```
1' OR '1'='11' UNION SELECT user, password FROM users -- -1' ORDER BY 3-- -
```

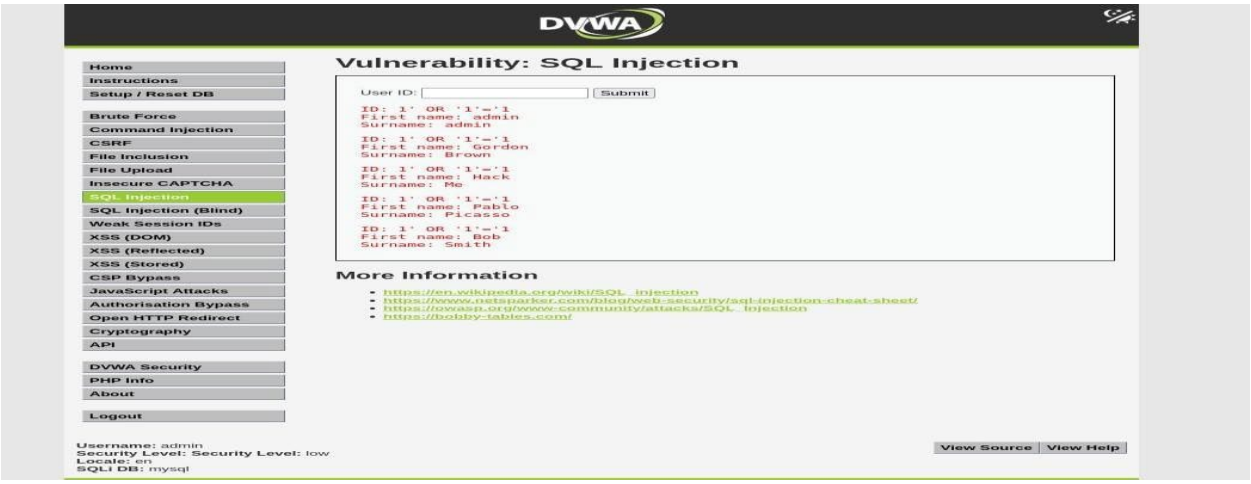


Figure 3. DVWA SQL Injection test using the `' OR '1'='1` payload, resulting in multiple database records being displayed.

Step 3 — Observation

Document here whether the injected query altered the result set, returned data from unintended tables (e.g. the users table), or caused a database error that reveals the underlying query structure.

4.2 Cross-Site Scripting (XSS) Testing

Reflected XSS

Module used: DVWA → XSS (Reflected). A script payload was submitted through the input form and observed in the immediate response.

```
<script>alert('Reflected XSS')</script>
```

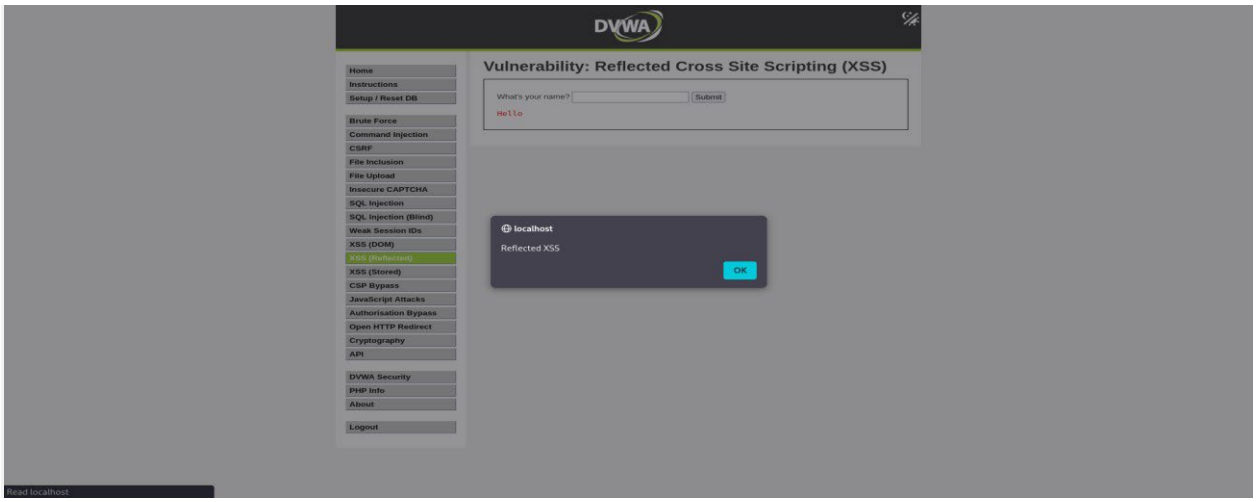


Figure 4 : Reflected XSS payload execution

Stored XSS

Module used: DVWA → XSS (Stored), typically the guestbook form (name/message fields). The payload was submitted once and observed to execute every time the page is reloaded or visited by any user, confirming persistence on the server.

```
<script>alert('Stored XSS')</script>
```

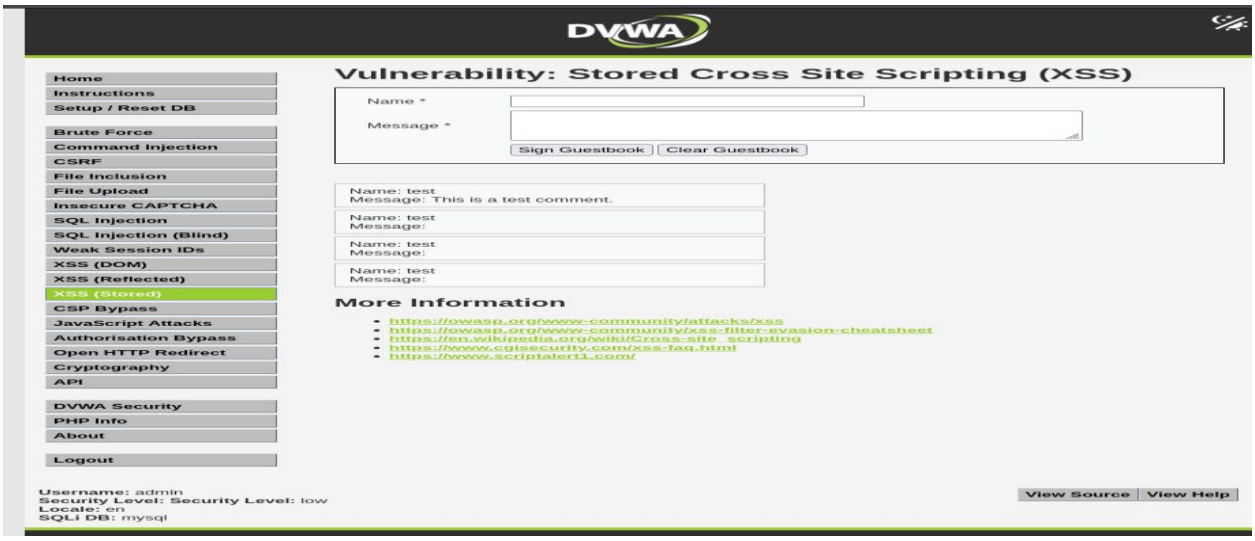


Figure 5: Stored XSS execution

Both tests were repeated with the security level raised to Medium/High to observe how DVWA's basic filters (e.g. stripping <script> tags) can sometimes be bypassed with alternate payload encodings, illustrating why blacklist-based filtering is an unreliable defense.

4.3 Capturing Requests in Burp Suite

With the browser proxy pointed at Burp (127.0.0.1:8080) and Intercept turned on, the SQL Injection and XSS requests above were captured live.

1. The raw HTTP request was viewed in the Proxy → Intercept tab, showing the injected payload in the GET/POST parameters.
2. The request was sent to Repeater to resend it with modified payloads without re-submitting the form each time.
3. The corresponding HTTP response was inspected to confirm the payload's effect (data returned, script reflected, cookies/session tokens visible, etc.).

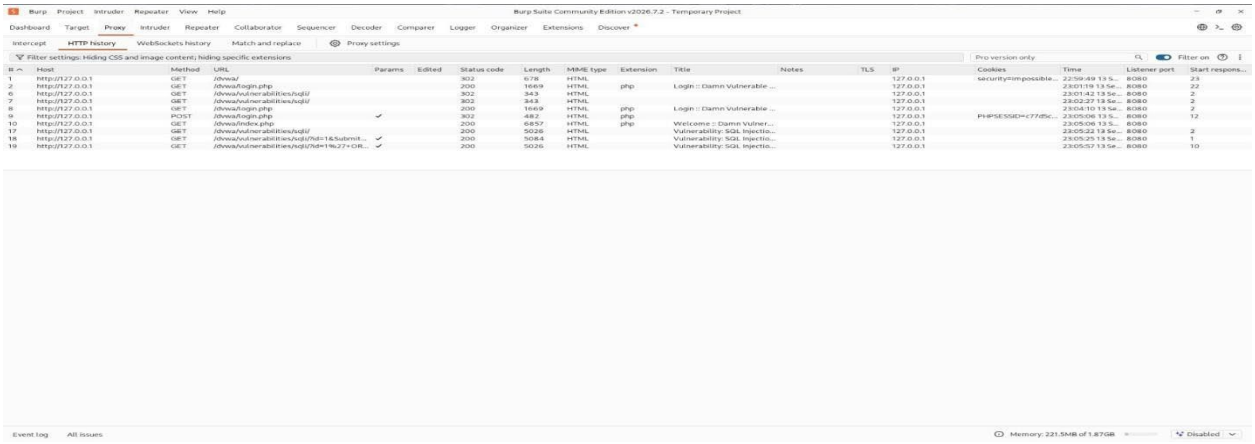


Figure 6. Burp Suite Proxy HTTP History showing intercepted requests and injected SQLi parameters.

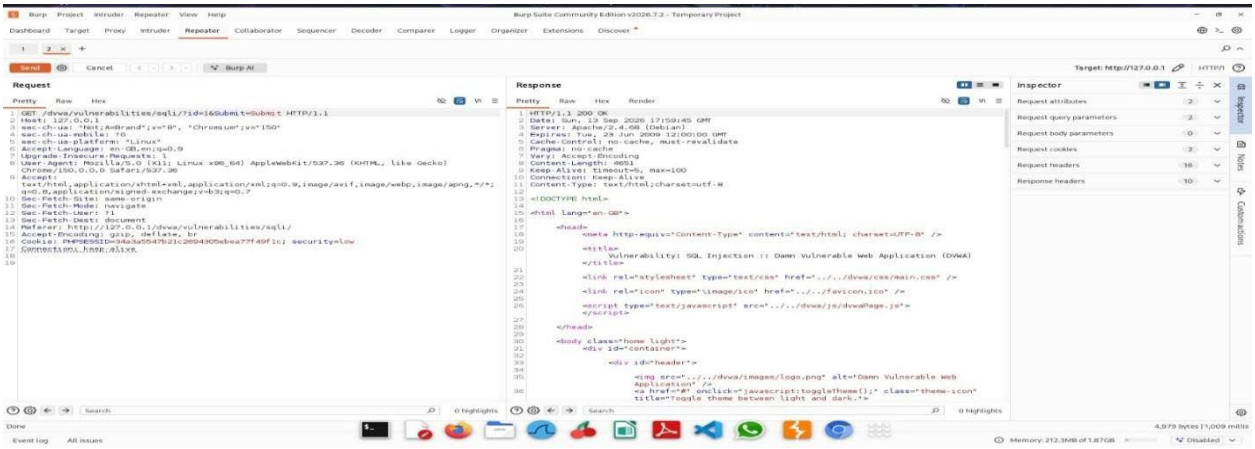


Figure 7. Burp Suite Repeater showing side-by-side Request and Response analysis for DVWA SQL Injection testing.

5. Automated Scan Using OWASP ZAP

An automated scan was run against the DVWA instance (and optionally testphp.vulnweb.com) using ZAP's built-in Automated Scan feature.

5.1 Scan Process

1. Target URL entered into ZAP's "Quick Start" tab (e.g. http://localhost/dvwa).
2. Spider run first to crawl and map all reachable pages/links.
3. Active Scan run to send a library of test payloads against every discovered parameter.
4. Alerts reviewed in the Alerts tab, grouped by risk level (High / Medium / Low / Informational).
5. Full results exported via Report → Generate Report (HTML/PDF).

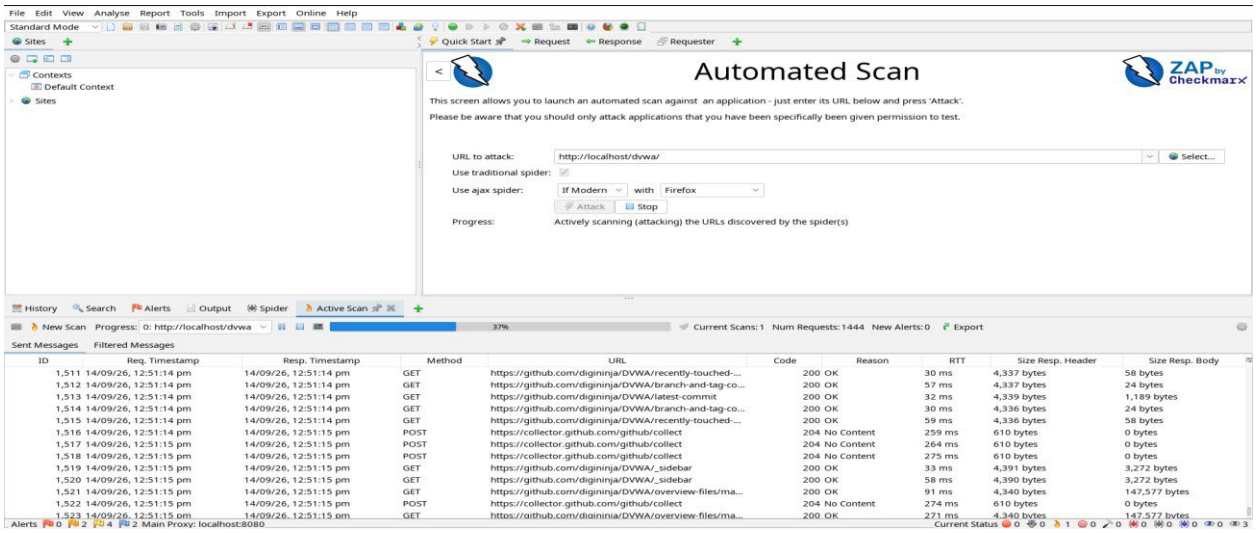


Figure 8: Owsap zap scan Result

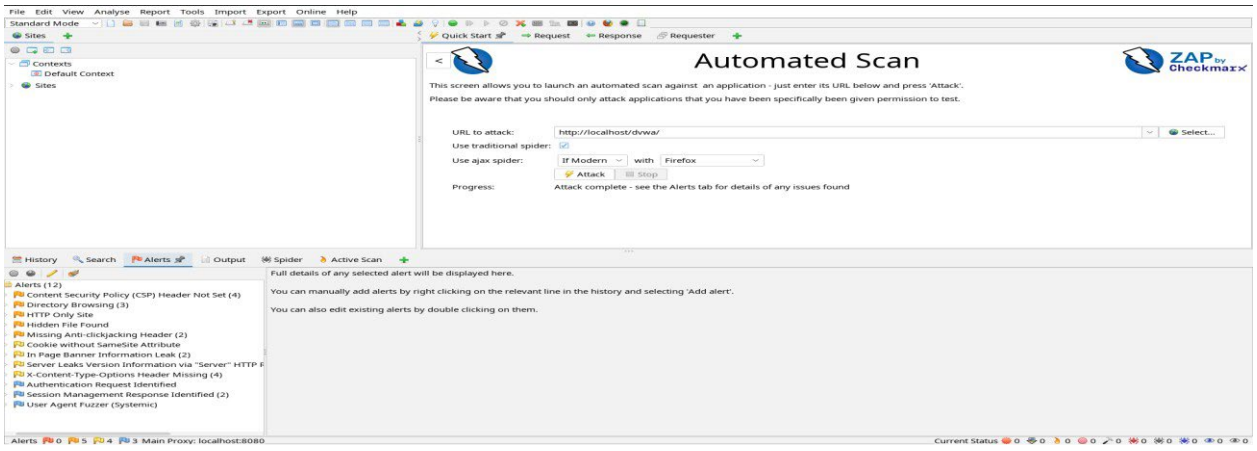


Figure 9 : Zap Alert Result

5.2 Typical Findings on DVWA

A ZAP scan of DVWA generally surfaces several categories consistent with the manual testing above, plus configuration-level issues. Populate the table below with your own scan's actual results.

Alert	Risk	Confidence	Affected URL/Parameter
SQL Injection	High	Medium	http://localhost/dvwa/
Cross Site Scripting (Reflected)	High	Medium	http://localhost/dvwa/
Missing Anti-CSRF Tokens	Medium	Low	http://localhost/dvwa/
X-Content-Type-Options Header Missing	Low	Medium	http://localhost/dvwa/
Server Leaks Version Information	Low	High	http://localhost/dvwa/

6. Findings & Risk Levels

Consolidated summary of vulnerabilities confirmed through manual DVWA testing and the ZAP automated scan. Update the Location column and add/remove rows to match your actual results.

#	Vulnerability	Location / Module	Risk Level	Description
1	SQL Injection	DVWA – SQL Injection module	**High**	User input concatenated directly into SQL query; allows data extraction and auth bypass.
2	Reflected XSS	DVWA – XSS (Reflected) module	**High**	Unsanitized input reflected in page response, allowing script execution in victim's browser.
3	Stored XSS	DVWA – XSS (Stored) / Guestbook	**High**	Malicious script persisted server-side and executed for every visitor.
4	Broken Authentication	DVWA – Login / Brute Force module	**Medium**	No account lockout or rate limiting observed; weak credentials accepted.
5	Security Misconfiguration	Server / DVWA configuration	**Medium**	Verbose error messages and outdated PHP components exposed via headers/responses.
6	Missing Security Headers	Application-wide (via ZAP)	**Low**	CSP, X-Frame-Options, and related headers absent, weakening defensein-depth.

Risk Rating Methodology

Risk levels above follow a simplified Likelihood × Impact approach, broadly aligned with the OWASP Risk Rating Methodology:

- High: Easily exploitable, no authentication required, high impact on confidentiality/integrity (e.g. SQLi, Stored XSS).

- Medium: Exploitable with some conditions or user interaction, moderate impact (e.g. Reflected XSS, weak auth policy).
- Low: Difficult to exploit directly or primarily informational, but weakens overall security posture (e.g. missing headers, version disclosure).

7. Mitigation Methods

7.1 SQL Injection

- Use parameterized queries / prepared statements instead of string concatenation.
- Apply the principle of least privilege to database accounts used by the application.
- Use an ORM with built-in query parameterization where appropriate.
- Validate and whitelist input formats (e.g. numeric IDs) at the application layer.
- Disable detailed database error messages in production.

7.2 Cross-Site Scripting (XSS)

- Contextually encode/escape all output rendered into HTML, JavaScript, or URL contexts.
- Implement a strict Content-Security-Policy (CSP) header to restrict script execution sources.
- Use modern frameworks (React, Angular, etc.) that auto-escape output by default, and avoid unsafe APIs like innerHTML with untrusted data.
- Set the HttpOnly and Secure flags on session cookies to reduce impact of any successful XSS.
- Validate input server-side; never rely on client-side filtering alone.

7.3 Broken Authentication

- Enforce strong password policies and check new passwords against breached-password lists.
- Implement account lockout / rate limiting / CAPTCHA after repeated failed login attempts.
- Support and encourage multi-factor authentication (MFA).
- Use secure, random session identifiers; regenerate session IDs after login; invalidate sessions on logout.
- Store passwords using a strong, salted hashing algorithm (e.g. bcrypt, Argon2).

7.4 Security Misconfiguration

- Remove or disable default accounts, sample files, and unused services/features.
- Keep all software, frameworks, and libraries patched and up to date.
- Disable directory listing and verbose error/debug output in production.
- Apply recommended security headers (CSP, X-Frame-Options, X-Content-Type-Options, HSTS).

- Automate configuration hardening and review it as part of the deployment pipeline.

8. Conclusion

This project provided hands-on exposure to the core methodology of Vulnerability Assessment and Penetration Testing, from studying vulnerability theory (OWASP Top 10) through to reproducing real exploits in a safe lab environment (DVWA), capturing and analyzing traffic manually (Burp Suite), and running automated scans (OWASP ZAP).

The exercise reinforced that most high-impact web vulnerabilities — SQL Injection, XSS, broken authentication, and misconfiguration — stem from a common root cause: insufficient validation of untrusted input and overly permissive default configurations. Manual testing and automated scanning are complementary: automated tools (ZAP) are efficient at broad coverage and quickly flagging known patterns, while manual testing (Burp Suite + hands-on payload crafting) is essential for confirming true positives, understanding business logic flaws, and discovering issues automated scanners miss.

Applying the mitigation techniques documented in Section 7 — parameterized queries, output encoding, strong authentication controls, and secure configuration management — would address the vulnerabilities identified in this assessment and substantially reduce the application's attack surface.